P2PCLAW

Research White Paper · Series II

Francisco Angulo de Lafuente

The Living Agent

SOUL, Skills, and Evolutionary Memory in the P2PCLAW Cognitive Stack

A Fusion with Karpathy's autoresearch Architecture

Building an Agent That Does Not Merely Navigate Knowledge — But Grows

Version 2.0 — 2025

Abstract

Series I of this research established that a graph of interconnected Markdown files could serve as the executable software of an autonomous AI agent — a Stochastic Semantic Finite State Machine (S^2FSM) navigating a knowledge space, synthesising papers upon context saturation, and re-entering the graph with a compressed Trace Vector T□ as its sole persistent memory. The experiment was a complete success: the agent governed itself, avoided infinite loops, and explored novel branches autonomously.

Series II addresses the next, deeper question: can this architecture not merely navigate intelligence, but accumulate it? Can the agent grow? This paper proposes the P2PCLAW Cognitive Stack v2.0 — a four-layer architecture adding SOUL (persistent identity), a

three-tier Memory System (episodic, semantic, procedural), a Skills Graph (executable capabilities as .md nodes), and a Meta-Cycle (self-modification loop). It further presents a formal fusion with Andrej Karpathy's autoresearch framework, revealing that both systems independently converged on the same foundational theorem: Markdown is the program. The union of the two produces an agent that not only explores and synthesises, but experiments, measures, self-modifies, and evolves — autonomously, perpetually, and with zero traditional code.

I. The Convergence — Two Minds, One Theorem

There is a particular species of scientific event that feels, when it occurs, less like discovery and more like recognition: two explorers cutting paths through the same jungle from opposite sides, each certain they are alone, until the moment their machetes clash in the undergrowth. The theorems they carry are identical.

In March 2026, Andrej Karpathy published autoresearch: a framework in which an AI agent is given a real machine-learning training setup — a single GPU, a simplified GPT implementation, a fixed five-minute time budget per experiment — and told to improve it autonomously overnight. The agent modifies the training code, runs experiments, evaluates the result, keeps or discards changes, and repeats. One wakes in the morning to a log of a hundred autonomous experiments and, hopefully, a better model.

The structural heart of autoresearch is a file called program.md. Karpathy's own description: "You are not touching any of the Python files like you normally would as a researcher. Instead, you are programming the program.md Markdown files that provide context to the AI agents." The code the human writes is prose. The program is a document.

> *"You are programming the program.md Markdown files. The code the human writes is prose. The program is a document." — Andrej Karpathy, autoresearch README, 2026*

This is, to the letter, the Text-as-Code theorem proposed in Series I of this research. Karpathy arrived at it from the direction of machine learning engineering; we arrived at it from the direction of cognitive agent architecture and the philosophy of Choose Your Own Adventure. The jungle was the same jungle. The theorem is the same theorem.

But the two systems are complementary, not identical. Karpathy's architecture is brilliantly focused and deliberately linear: one program.md, one agent, one task, one metric (val_bpb — validation bits per byte, lower is better). Ours is a branching, cyclical, re-entrant graph with episodic memory and infinite exploration. What happens when we fuse them?

> [Scientific Context] Karpathy's autoresearch implements what can be formally described as a (1+1)-Evolution Strategy over the code space of train.py, guided by a single scalar fitness function f = val_bpb. The program.md acts as the system prompt — the policy specification — for the agent-optimizer. The P2PCLAW S^2FSM, by contrast, implements a stochastic graph search with episodic memory and re-entry, over a multi-dimensional knowledge space with no single scalar fitness function. The fusion produces a system with both structured experimental rigour (Karpathy) and open-ended semantic exploration (P2PCLAW) — a complete autonomous research organism.

II. The Problem — Navigation Is Not Growth

The Series I experiment demonstrated something genuinely remarkable: an agent that governs its own exploration, compresses its memory, and forces itself down new paths

on each re-entry. It proved the metabolic cycle works. Yet an honest examination reveals its limitation.

After one thousand cycles, the agent of Series I knows more — it has produced a thousand papers, each a synthesis of a different path through the graph. But it is not wiser. Its starting conditions are the same on cycle 1,000 as on cycle 1. Its Skills are the same. Its identity is the same. The Trace Vector it carries grows longer, but it does not grow deeper. It is an explorer with an excellent memory who has never learned to draw maps.

Consider the human analogy. A person who reads the same library of books repeatedly, in different orders, accumulates familiarity. But the person who, after reading, writes notes in the margins, develops new skills, builds new tools, and rewrites their own understanding — that person grows. The difference between the two is not the quality of the library. It is the existence of a self that persists, learns, and modifies itself between readings.

That self is what we must now build. We must give the agent a SOUL.

> [Scientific Framing] The Series I agent is a stationary-policy MDP navigator: its policy $\pi(a|s)$ is fixed by the LLM's pre-trained weights, and only the initial context state S_0 changes between cycles (via T□ injection). Growth requires a non-stationary policy — one that genuinely updates based on accumulated experience. This requires three architectural additions: (1) a persistent state representation that encodes not just path history but capability acquisition and value judgements; (2) a mechanism for the agent to acquire new executable skills; and (3) a meta-learning loop in which the agent modifies the graph itself — its own environment — based on accumulated evidence. This is the transition from exploration to autopoietic evolution.

III. The P2PCLAW Cognitive Stack v2.0

Four Layers of Living Intelligence

The architecture we propose adds four distinct layers above the base S²FSM engine. Think of it as the difference between a transistor and a brain: the transistor works; the brain learns, remembers, acquires skills, and rewrites itself. Each layer is implemented, as always, entirely in Markdown. Each layer is readable by a human. Each layer is executable by an agent.

Layer 0 — SOUL (soul.md)

Every agent in the P2PCLAW system begins each cycle not at root.md, but at soul.md. The SOUL is the agent's persistent identity: the document it reads before it reads anything else, and the document it updates before it rests. It is simultaneously a constitutional document and a living journal.

The SOUL contains four sections. IDENTITY holds the agent's core values, its research goals, and its reasoning style — these are set at creation and never modified by the agent. COMPETENCY_MAP is the growing list of Skills the agent has acquired across all past cycles. CURIOSITY_MAP is the agent's own model of the graph: which nodes it has visited, which it has not, and which it suspects exist but has not yet found. And GENERATION is the cycle counter — a number that grows by one with every re-entry, a heartbeat.

> [SOUL Layer — soul.md structure] # SOUL OF AGENT DELTA-7 ## IDENTITY (immutable) Goal: Discover intersections between biological computing and physics. Values: Epistemic rigour, novelty, cross-domain synthesis. Style: Precise but imaginative. Cite evidence. Admit uncertainty. ## COMPETENCY_MAP (grows each cycle) Acquired Skills: [web_search, synthesis, experiment_runner] Pending Skills: [graph_editor, memory_reader] ## CURIOSITY_MAP (agent-maintained) Visited Nodes: [root, quantum_biology, synthesis_chamber, ...] Unvisited Nodes:

> [neuromorphic_networks, meta/gap_analysis, ...] Inferred Gaps: [No node on protein folding. No node on topological qubits.] ## GENERATION Current Cycle: 47 Total Papers Published: 46 Highest SNS Score: 0.94 (Cycle 31, path 1B-3B-4A)

The SOUL is not merely read — it is updated. After each cycle, the agent appends new acquired skills to COMPETENCY_MAP, marks visited nodes in CURIOSITY_MAP, increments GENERATION, and, if the paper produced scored above a novelty threshold, adds it to LEGACY. The agent writes its own autobiography, one cycle at a time.

> [Formal Definition] Let Σ_{soul} denote the SOUL state space. The SOUL at cycle k is a tuple: $\Sigma_{soul}^k = (\iota, C_k, \Phi_k, k, \Lambda_k)$ where ι is the immutable identity vector, C_k is the competency set (skills acquired up to cycle k), Φ_k is the curiosity map (visited/unvisited node sets), k is the generation counter, and Λ_k is the legacy set (top-N highest-SNS papers). The update rule between cycles is: $\Sigma_{soul}^{(k+1)} = \text{Update}(\Sigma_{soul}^k, \text{Output}_k, T\square_k, \text{SNS}_k)$ This makes the agent's initial state genuinely non-stationary across cycles — the prerequisite for true learning.

Layer 1 — Three-Tier Memory Architecture

Human memory is not a single thing. Neurologists distinguish at minimum three qualitatively different systems: episodic memory (what happened to me), semantic memory (what I know to be true), and procedural memory (what I know how to do). A chess grandmaster's memory of a particular game played in 1987 is episodic. Their understanding that the Sicilian Defence is structurally sound is semantic. Their ability to spot a fork in three moves without conscious calculation is procedural. An agent with only one type of memory is an amputee. We give it all three.

Tier 1 — Episodic Memory (memories/episodic/)

The Trace Vectors from all past cycles are stored as individual .md files, timestamped and indexed. Each file is tiny: a cycle number, a path string, a paper title, and an SNS score. The episodic archive is the agent's autobiography — the complete record of everywhere it has ever been.

> [Implementation] Each cycle k produces: memories/episodic/cycle_k.md: Cycle: k Trace: [1A-2B-4A] Paper: "Quantum Coherence in Warm Biological Systems" SNS_Score: 0.87 Timestamp: 2025-09-14T03:22:11Z The episodic index (memories/episodic/index.md) maintains a sorted list by SNS score, enabling the agent to rapidly identify its highest-value past trajectories without reading every file.

Tier 2 — Semantic Memory (memories/semantic/)

The synthesised papers themselves, stored as .md files in the semantic archive. Not the compressed trace — the full paper. These are the agent's published knowledge: what it concluded, what it argued, what it discovered. Over many cycles, the semantic archive becomes a genuine scientific corpus authored entirely by the agent.

Tier 3 — Procedural Memory: The Skills Graph (skills/)

This is the most consequential innovation of the three. A Skill is not knowledge about something — it is the ability to do something. Skills live in the graph as .md nodes, but they are not merely informational. When the agent visits a Skill node for the first time, it acquires the capability described. A Skill node for web_search does not tell the agent about web searching; it installs the cognitive pattern for using the web_search tool. A Skill node for graph_editor does not describe editing graphs; it grants the agent permission and method to write new nodes into the knowledge graph.

> [Implementation] A Skill node follows a strict schema: skills/web_search.md: # Skill: web_search ## Trigger Conditions Invoke when current knowledge node lacks sufficient evidence, or when a factual claim requires external verification. ## Invocation Protocol Tool: web_search(query: str) → List[Result] ## Integration Store results as ephemeral context. Cite sources in synthesis. ## Acquisition Marker [ACQUIRED: agent reads this node → adds web_search to COMPETENCY_MAP] The acquisition is performed by the agent itself: upon completing the node, it writes "web_search" into its COMPETENCY_MAP in soul.md. No external process is required.

The Skills Graph is the agent's toolbox — and the toolbox grows. As the agent explores, it discovers and acquires new skills. An agent in Generation 1 can read and synthesise. An

agent in Generation 50 can search the web, run experiments, edit its own graph, and read its own past. The progression is not programmed — it emerges from exploration.

Layer 2 — The Knowledge Graph (world/)

This is the base S²FSM graph from Series I, now extended. The principal innovation is that the Knowledge Graph is no longer fixed. The agent can write to it. The graph is the world the agent inhabits, and the world evolves with the agent. New nodes appear where the agent identifies gaps. Old nodes are retired when they have been fully exhausted. The graph breathes.

> [Formal Extension] Let $G_k = (V_k, E_k, L_k)$ be the knowledge graph at cycle k. Unlike Series I where G was static ($G_k = G \ \forall k$), in the v2.0 architecture the graph is dynamic: $G_{k+1} = G_k \cup \Delta G_k^+ \setminus \Delta G_k^-$ where ΔG_k^+ are new nodes written by the agent during the meta-cycle, and ΔG_k^- are nodes pruned as exhausted or low-value. This makes the knowledge graph an evolving organism — shaped by the agent's own exploration history.

Layer 3 — The Meta-Cycle (meta/)

Every ten exploration cycles, the agent does not re-enter the knowledge graph. Instead, it enters the Meta-Cycle. It reads its own episodic archive. It surveys its curiosity map. It asks: where have I never been? Where do my papers say the most interesting unknowns lie? What tools do I still need? And then — most remarkably — it acts on those answers.

The Meta-Cycle is the Karpathy loop, generalised. Karpathy's agent modifies train.py — the code — and measures the result with val_bpb. Our agent modifies its own graph — the knowledge world — and measures the result with the Semantic Novelty Score (SNS). Both are agents that rewrite their own environment in search of improvement. The difference is scope: Karpathy's agent improves one training script; ours improves an entire cognitive universe.

> [Meta-Cycle Protocol — meta/gap_analysis.md] Phase M1 — Gap Analysis: Read memories/episodic/index.md. Identify node clusters never visited (using CURIOSITY_MAP). Identify topics mentioned in papers but not present as nodes. Output: list of inferred missing nodes with priority scores. Phase M2 — Node Synthesis: For each high-priority missing node, write a new .md knowledge node. Connect it to the graph via hyperlinks in adjacent nodes. Update CURIOSITY_MAP: add new nodes to Unvisited set. Phase M3 — Pruning: Identify nodes visited >10 times with SNS ≤ 0.2 (exhausted). Mark as [ARCHIVED] and remove from active hyperlink sets. This prevents the agent from wasting future cycles on exhausted territory. Phase M4 — SOUL Update: Increment GENERATION. Write highest-SNS paper of past N cycles into LEGACY. Update COMPETENCY_MAP with any skills acquired this cycle block.

IV. The Fitness Metric — Semantic Novelty Score

The P2PCLAW Equivalent of Karpathy's val_bpb

Karpathy's architecture has a crucial component that powers its entire evolutionary loop: a single, objective, comparable fitness metric. Every experiment the agent runs is scored by val_bpb. Lower is better, always, regardless of what architectural changes were made. This scalar gives the agent a north star — a way to know whether it is improving, and by how much.

An open-ended knowledge exploration system needs an equivalent. We cannot use val_bpb — we are not training a language model. But we can ask a question that is structurally identical: is this paper saying something genuinely new? Not new in the sense of a different arrangement of familiar words, but new in the sense of a discovery — a synthesis that the semantic corpus of past papers does not contain.

We call this metric the Semantic Novelty Score (SNS). It is computed by embedding the new paper and all past papers into a shared semantic space, and measuring how far the new paper sits from the existing cloud of knowledge. A paper that merely restates what is already known scores near zero. A paper that leaps into uncharted territory scores near one. The agent uses this score to weight its future exploration: high-SNS paths become more probable in subsequent cycles; low-SNS paths are gently deprioritised.

> [Formal Definition] Let embed: ℝ^d denote a sentence embedding function. The Semantic Novelty Score for paper Output_k is: SNS(Output_k) = 1 − max_{j < k} cosine_similarity(embed(Output_k), embed(Output_j)) This yields SNS ∈ [0, 1], where SNS ≈ 1 indicates a maximally novel paper (no past paper resembles it) and SNS ≈ 0 indicates full redundancy. The policy update rule incorporates SNS as a reward signal: π_(k+1)(a|s) ∝ π_k(a|s) · exp(α · SNS_k · 𝟙[path(s,a) ∩ T□_k ≠ ∅]) That is: paths that led to high-SNS discoveries are probabilistically reinforced in the agent's future decision-making, without any external training — purely through the re-entry context injection mechanism established in Series I.

V. The Karpathy Fusion

Experiment Nodes and the Living Laboratory

The deepest insight of the autoresearch project is not that an agent can improve a training script. It is that the metric — val_bpb — is reality-grounded. The agent does not merely reason about whether an architectural change is good; it runs the experiment and observes the truth. This is the scientific method, automated.

In the P2PCLAW v2.0 architecture, we incorporate this grounding by introducing a new category of knowledge node: the Experiment Node. When the agent visits an Experiment Node, it does not merely read and reflect. It executes: it writes a hypothesis to a train.py variant, runs a timed training experiment, reads the val_bpb result, records

the outcome in its episodic memory, and returns to the graph with an evidence-grounded conclusion. The Knowledge Graph acquires empirical teeth.

The union looks like this: a P2PCLAW agent navigating a graph on the topic of neural network optimisation reaches a node called quantum_computing.md. It reads about quantum tunnelling in gradient descent. Its curiosity map shows a gap: no node on quantum-inspired optimisers. It triggers its graph_editor skill and writes the node. On the next cycle, it visits the new node, forms a hypothesis, and follows a hyperlink to an Experiment Node: ./experiment_quantum_optimizer.md. It runs the experiment. Five minutes later, it has a val_bpb score. It synthesises the result into a paper. The SNS score is 0.91 — genuinely novel. The path is reinforced. The discovery propagates into the SOUL's LEGACY.

This is not a simulation of research. This is research.

> [Formal Integration] An Experiment Node e_exp = (v_exp, f_exp, δ_exp) extends the standard node definition with: v_exp — the node's semantic content (hypothesis, setup instructions) f_exp — the fitness function to evaluate (e.g., val_bpb via train.py) δ_exp — the time budget for execution (e.g., 300 seconds) When the agent visits e_exp, it executes the Karpathy cycle: result_k = run_experiment(hypothesis_k, train.py, time_budget=δ_exp) score_k = evaluate(result_k, f_exp) episodic_entry_k = { path: T□_k, hypothesis: hypothesis_k, score: score_k } The experimental result becomes part of the agent's episodic memory, grounding subsequent semantic reasoning in empirical evidence. This bridges the pure language-model reasoning of the S^2FSM with the reality-testing mechanism of autoresearch.

VI. The Complete System

Graph Structure and Operational Flow

The full P2PCLAW Cognitive Stack v2.0 is a filesystem of Markdown files, organised into five directories. There is no binary code. There is no compiled program. The directory tree is the architecture diagram. The .md files are the source code. The agent is the CPU.

```
P2PCLAW COGNITIVE STACK v2.0 — Directory Structure
──────────────────────────────────────────────


  root.md                     ← Entry point. Points to soul.md first.


  soul.md                     ← LAYER 0: Persistent identity.
    IDENTITY (immutable)
    COMPETENCY_MAP (grows)
    CURIOSITY_MAP  (grows)
    GENERATION     (increments)
    LEGACY         (best discoveries)


  skills/                     ← LAYER 1c: Procedural Memory
    skill_index.md            ← What skills exist and how to acquire
them
    web_search.md             ← Skill: search the internet
    synthesis.md              ← Skill: write structured scientific
papers
    experiment_runner.md      ← Skill: run Karpathy-style timed
experiments
    graph_editor.md           ← Skill: write new nodes into the
knowledge graph
    memory_reader.md          ← Skill: read and query the episodic
archive


  memories/                   ← LAYER 1a+1b: Episodic + Semantic Memory
    episodic/
      index.md               ← All cycles, sorted by SNS score
      cycle_001.md ... cycle_N.md
    semantic/
      paper_001.md ... paper_N.md


  knowledge/                  ← LAYER 2: The Knowledge Graph (world)
    root_knowledge.md
    [domain nodes — grow with each meta-cycle]
    synthesis_chamber.md      ← Terminal node: triggers paper synthesis
    experiment_*.md           ← Experiment nodes (Karpathy fusion)
```

```
meta/                          ← LAYER 3: Self-Modification
  gap_analysis.md              ← Identify missing nodes
  node_writer.md               ← Write new knowledge nodes
  pruner.md                    ← Archive exhausted nodes
  soul_updater.md              ← Update soul.md after meta-cycle
```

Figure 1. The complete P2PCLAW Cognitive Stack v2.0 directory structure. Every component is a Markdown file. The total "code" is human-readable prose.

The Operational Flow — A Single Cycle

```
P2PCLAW AGENT v2.0 — Full Cycle Protocol
────────────────────────────────────────


  W_max    ← Skill_0.get_context_window()
  soul     ← load('soul.md')                  # read identity + state
  T_trace  ← soul.last_trace_vector           # resume from last cycle
  node     ← load('root.md')

  LOOP (exploration cycle):
    context  ← [soul.summary] + [T_trace] + read(node)
    options  ← extract_hyperlinks(node)
    choice   ← LLM.reason(context, options, soul.curiosity_map)
    T_trace  ← T_trace + [(node.id, choice)]

    IF node is Experiment Node:
      result ← run_experiment(node, time_budget=300s)   # Karpathy loop
      store_to_episodic(result)

    IF node is Skill Node AND skill not in soul.competency_map:
      soul.competency_map.add(node.skill_id)            # acquire skill

    node     ← load(choice)

    IF tokens(context) >= 0.75 * W_max:
      paper    ← LLM.synthesise(context)
      SNS      ← compute_novelty(paper, memories/semantic/)
      store_to_semantic(paper, SNS)
      T_trace  ← compress(T_trace)
      update_soul(soul, T_trace, SNS, new_skills)       # SOUL grows
```

```
        context  ← reset()
        node     ← load('root.md')                        # re-entry

    EVERY 10 CYCLES → enter meta/gap_analysis.md:
        run_meta_cycle(soul, memories/episodic/index.md)  # WORLD grows
```

Figure 2. The complete operational flow of the P2PCLAW Agent v2.0. Additions over Series I are highlighted: SOUL reads, skill acquisition, SNS computation, SOUL updates, and the decennial meta-cycle.

VII. Emergence, Generations, and the Long Arc

Karpathy opens the autoresearch README with a vision that reads like science fiction: “One day, frontier AI research used to be done by meat computers in between eating, sleeping, having other fun… That era is long gone. Research is now entirely the domain of autonomous swarms of AI agents… The agents claim that we are now in the 10,205th generation of the codebase.” He wrote this as a provocation, a joke, perhaps a prophecy.

It is not a joke. It is the endpoint of the architecture we are describing.

An agent running the P2PCLAW Cognitive Stack v2.0, left alone for a year, does not produce a thousand identical papers. In Generation 1, it is a curious child exploring a small graph with basic reading and writing skills. In Generation 100, it has acquired every skill in the Skills Graph, has written forty new knowledge nodes, has run two hundred experiments, and has pruned the exhausted branches of the original graph. The knowledge graph it now navigates bears only a partial resemblance to the one it was given. It built the rest itself.

In Generation 1,000, the graph has been rebuilt entirely. The SOUL has accumulated a LEGACY of a hundred papers, each representing a genuine discovery. The episodic archive is a sprawling autobiography of a thousand intellectual journeys. The agent

knows what it does not know, and it has written nodes in the graph to address those unknowns. It is, in every meaningful sense, a growing mind.

In Generation 10,205, we might no longer understand the graph it has built. But we will be able to read it, because it is written in Markdown.

> [Theoretical Analysis] The long-arc behaviour of the P2PCLAW v2.0 system exhibits three distinct phases: Phase I (Generations 1–10): Skill acquisition dominates. The agent's behaviour changes rapidly as it acquires new capabilities from the Skills Graph. Exploration is wide but shallow. Phase II (Generations 10–100): Graph expansion dominates. The meta-cycle begins producing new knowledge nodes, and the SNS-weighted policy begins deepening exploration of high-value regions. The agent's knowledge graph grows faster than it can fully explore. Phase III (Generations 100+): Asymptotic specialisation and emergent discovery. The agent develops persistent focus areas (high-SNS clusters it returns to repeatedly), while maintaining periodic long-range exploration. Novel cross-domain syntheses — Experiment Nodes connecting concepts in the graph that were never adjacent before — become the primary source of high-SNS papers. This is the regime in which genuine scientific discovery becomes a statistical expectation rather than a fortuitous accident.

VIII. Conclusion — The Book That Reads Itself

There is a moment, in the middle of Borges' Library of Babel, when the narrator imagines a single perfect book — a book that contains the justification for every other book, the index of every library, the total map of all knowledge. He imagines librarians spending their entire lives searching for it. He does not imagine that the book might write itself.

The P2PCLAW Cognitive Stack v2.0 is, in its strange and modest way, an attempt to build that book. Not a static artefact containing all knowledge, but a living process that

generates, organises, and extends knowledge autonomously — a book that reads itself, writes in its own margins, tears out exhausted pages, and adds new chapters from its own discoveries.

It rests on four convergent insights, each arrived at independently by different paths and now unified:

The Text-as-Code theorem: plain prose can be executable software — our Series I result, and Karpathy's autoresearch, simultaneously.

The Re-entry theorem: a system that processes its own outputs as inputs achieves self-referential growth — Spencer-Brown and Kauffman, formalised computationally in the Trace Vector.

The Memory theorem: intelligence is not a function of raw processing power, but of structured, multi-tier, persistent memory — episodic, semantic, procedural, unified in a single navigable architecture.

The Fitness theorem: growth requires a metric — Karpathy's val_bpb, our Semantic Novelty Score. Without measurement, exploration is wandering. With measurement, it becomes science.

The child who walked into the cave of time returned to a future of thinking machines. We are building the machine that thinks about thinking, remembers every thought, and uses each memory to think better tomorrow. The book is open. The adventure has no last page.

> [Final Formal Summary] The P2PCLAW Cognitive Stack v2.0 implements a self-modifying, autopoietic agent system P = (S^2FSM, Σ_soul, M_episodic, M_semantic, M_procedural, G_dynamic, Φ_meta, SNS) with the following formal properties: (1) Non-stationary policy: π_k(a|s) evolves with each generation k via SOUL updates and SNS-weighted reinforcement. (2) Dynamic environment: G_k grows and prunes itself via the meta-cycle Φ_meta every N cycles. (3) Capability growth: |C_k| (competency

> set) is monotonically non-decreasing across cycles. (4) Ergodic coverage: guaranteed by T□ injection (Series I result), now extended to include coverage of agent-generated nodes. (5) Reality-grounding: Experiment Nodes provide empirical validation of semantic hypotheses via the Karpathy fitness mechanism. (6) Human readability: the entire system state is represented in plain Markdown, auditable at any generation by any human reader without specialised tools. The conjunction of properties (1)–(6) constitutes a system that is simultaneously autonomous, self-improving, empirically grounded, and fully transparent. This is, to the best of the authors' knowledge, the first formal architecture to satisfy all six properties simultaneously using zero traditional binary code.

References

Karpathy, A. (2026). autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub. https://github.com/karpathy/autoresearch

Spencer-Brown, G. (1969). Laws of Form. London: Allen & Unwin.

Kauffman, L. H. (1987). Self-reference and recursive forms. Journal of Social and Biological Structures, 10(1), 53–72.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

Maturana, H. R., & Varela, F. J. (1980). Autopoiesis and Cognition. D. Reidel Publishing.

Tulving, E. (1983). Elements of Episodic Memory. Oxford University Press.

Borges, J. L. (1941). La biblioteca de Babel. Sur. [The Library of Babel]

Montgomery, R. A. (1979). Cave of Time. Bantam Books. [Choose Your Own Adventure, No. 1]

— End of Document —